

AI ASSISTED CODING

ASSIGNMENT - 19.1

ROLL NO. 2303A510J0

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

Python code:

```
class Employee:
    def __init__(self, name, salary, department):
        self.name = name
        self.salary = salary
        self.department = department

    def give_raise(self, amount):
        self.salary += amount
        return self.salary
```

```
def display_info(self):  
    return f"Employee: {self.name}, Salary:  
{self.salary}, Department: {self.department}"  
  
emp = Employee("Rohit", 50000, "Engineering")  
print(emp.display_info())  
emp.give_raise(5000)  
print(emp.display_info())
```

Output:

```
Employee: Rohit, Salary: 50000, Department:  
Engineering  
Employee: Rohit, Salary: 55000, Department:  
Engineering
```

Prompt:

I have a Python class and I need to convert it into Java.
Generate a fully working Java class with:

- proper constructor
- correct data types
- equivalent attributes and methods
- a main() method to run the class
- an output that matches the Python version

Here is the Python code:
(class above)

Convert it and show the expected output.

Code:

```
public class Employee {
    private String name;
    private double salary;
    private String department;

    public Employee(String name, double salary,
String department) {
        this.name = name;
        this.salary = salary;
        this.department = department;
    }

    public double giveRaise(double amount) {
        this.salary += amount;
        return this.salary;
    }

    public String displayInfo() {
        return "Employee: " + name + ", Salary: " +
salary + ", Department: " + department;
    }

    public static void main(String[] args) {
        Employee emp = new Employee("Rohit", 50000,
"Engineering");
        System.out.println(emp.displayInfo());
        emp.giveRaise(5000);
    }
}
```

```
        System.out.println(emp.displayInfo());
    }
}
```

Output:

```
Employee: Rohit, Salary: 50000.0, Department:
Engineering
Employee: Rohit, Salary: 55000.0, Department:
Engineering
```

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

C++ Code:

```
#include <iostream>
using namespace std;

int factorial(int n) {
```

```

    if(n <= 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    cout << "factorial(3) = " << factorial(3) <<
endl;
    cout << "factorial(5) = " << factorial(5) <<
endl;
    cout << "factorial(7) = " << factorial(7) <<
endl;
    return 0;
}

```

Output (C++):

```

factorial(3) = 6
factorial(5) = 120
factorial(7) = 5040

```

Prompt:

Translate the following recursive C++ factorial function into Python.

Make sure the Python version:

- Uses recursion like the original
- Follows Pythonic style
- Produces the same output

Also run the Python version for inputs 3, 5, and 7.

Python Code:

```
def factorial(n):  
    if n <= 1:  
        return 1  
    return n * factorial(n - 1)  
  
print("factorial(3) =", factorial(3))  
print("factorial(5) =", factorial(5))  
print("factorial(7) =", factorial(7))
```

Output (Python):

```
factorial(3) = 6  
factorial(5) = 120  
factorial(7) = 5040
```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of

data.

SQL Code:

```
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

Output (SQL Expected Result):

name	salary
Rohit	72000
Meera	85000

Prompt:

Convert the following SQL query into an equivalent Pandas DataFrame operation.

Make sure the generated code:

- Uses a sample employees DataFrame
- Applies the same filtering condition
- Selects the same columns
- Produces the same final output as the SQL query

SQL query:

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

Generate the Pandas version and show the output.

Python Code (Pandas Equivalent):

```
import pandas as pd

# Sample DataFrame
data = {
    "name": ["Rohit", "Meera", "Amit", "Sara"],
    "salary": [72000, 85000, 45000, 30000]
}

df = pd.DataFrame(data)

# Pandas equivalent of the SQL query
result = df[df["salary"] > 50000][["name",
"salary"]]

print(result)
```

Output (Pandas):

	name	salary
0	Rohit	72000
1	Meera	85000

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will choose a real-time use case where they need code in

multiple languages. Example:

- Converting a Python web scraping snippet into JavaScript

for browser automation.

- Translating a Java method into C# for enterprise applications.

Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages.

Python Code (Original Snippet):

```
import requests
from bs4 import BeautifulSoup

def get_title(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.text,
"html.parser")
    return soup.title.string

print(get_title("https://www.python.org"))
```

Output (Python Expected Result):

```
Welcome to Python.org
```

Prompt:

Translate the following Python web-scraping snippet into an equivalent JavaScript version that runs in a browser environment using `fetch()` and `DOMParser`.

Make sure both implementations:

- Extract the page title
- Produce the same output
- Use appropriate libraries for each language

After converting, show the JavaScript code and its expected output, followed by a short list of challenges faced during translation.

Python snippet:

```
import requests
from bs4 import BeautifulSoup
```

```
def get_title(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.text, "html.parser")
    return soup.title.string
```

JavaScript Code (Converted Version):

```
async function getTitle(url) {
    const response = await fetch(url);
    const html = await response.text();
    const parser = new DOMParser();
    const doc = parser.parseFromString(html,
    "text/html");
    return doc.querySelector("title").innerText;
}
```

```
getTitle("https://www.python.org").then(title => {  
    console.log(title);  
});
```

Output (JavaScript Expected Result):

```
Welcome to Python.org
```

Translation Challenges (Documented Notes):

1. Python's requests + BeautifulSoup work synchronously, while JavaScript uses async fetch + DOMParser.
2. Exception handling differs (Python's try/except vs. JS Promises).
3. BeautifulSoup has convenient parsing helpers; DOMParser requires precise HTML queries.
4. Running JS scraping requires a browser environment or additional Node.js libraries.
5. Differences in indentation, typing, and language syntax must be adapted manually.